

Universidad de Ingeniería y Tecnología
2026-2

El Problema del Vendedor Viajante (TSP): Modelado y Estrategias de Optimización

Optimización (DS2011)

Integrantes

Cuadros Villegas, Denilson Jermai 202510232
Obregon Carhuavilca, Alondra Solange 202510394
Rosales Esteban, Antony Yonel 202510396
Timana Carmona, Jose Manuel 202510502

Docente: Torres Castillo, Victor Antonio

Barranco, 05 de septiembre de 2026

1. Introducción a los Problemas de Optimización Combinatoria

La optimización busca la mejor opción entre muchas posibles. Sus problemas se dividen en dos grupos: los de variables continuas, que pueden tomar cualquier valor, y los de variables discretas, llamados **combinatorios** [22]. En un problema combinatorio se busca el mínimo (o el máximo) de una función sobre un conjunto *finito* de soluciones S, sin exigir ninguna propiedad especial a la función ni a S. La dificultad está en el tamaño: S suele tener tantos elementos que revisarlos todos para hallar el óptimo es impracticable [19].

Algunos problemas conocidos del área son:

- **Problema de la mochila.** Se tiene una mochila de capacidad C y n objetos, cada uno con un valor p_i y un tamaño w_i. Hay que elegir qué objetos guardar para lograr el mayor valor posible sin superar la capacidad [25].
- **Problema de asignación generalizada (PAG).** Hay m agentes y n tareas, y C_{ij} es el costo de asignar la tarea j al agente i. Se busca repartir las tareas al menor costo total, respetando que cada agente tiene recursos limitados (tiempo, materia prima, capital, infraestructura). Es un problema NP-completo [11].
- **Problema Flow-Shop.** n tareas deben pasar por m máquinas en línea, siempre en el mismo orden. Se busca la secuencia de tareas que minimice C_{max}, el momento en que la última tarea sale de la máquina m [11].

El más emblemático de todos es el **Problema del Vendedor Viajante** (*Traveling Salesman Problem*, TSP), uno de los más estudiados en Investigación Operativa [20]. Su historia resume la del área:

- **Década de 1930:** matemáticos de Viena y Harvard, entre ellos Menger, lo formulan por primera vez. En 1937, Merrill Flood lo difunde en la comunidad matemática [20].
- **1954:** Dantzig, Fulkerson y Johnson resuelven un caso de 49 ciudades con planos de corte. Es el inicio del TSP como problema de optimización moderna [5].
- **1972:** con el desarrollo de la teoría de la complejidad, Karp demuestra que el TSP pertenece a los problemas NP-difíciles [14].

Desde entonces, el TSP es el banco de pruebas de casi toda técnica nueva para problemas enteros o combinatorios: ramificación y acotación, métodos poliédricos y procedimientos metaheurísticos [19].

2. El Problema del Vendedor Viajante (TSP)

El TSP se enuncia así: dado un conjunto de ciudades y las distancias entre ellas, encontrar la ruta más corta que visite cada ciudad exactamente una vez y regrese al punto de partida [21].

Formalmente, se modela con un grafo no dirigido $G = (V, E)$, donde $V = \{1, 2, \ldots, n\}$ son las ciudades (nodos) y $E = \{(i, j) \mid 1 \leq i < j \leq n\}$ son las conexiones entre ellas (aristas). Cada arista tiene un costo conocido C_{ij}, guardado en la matriz de distancias C. El objetivo es encontrar el **ciclo**

hamiltoniano —un ciclo que pasa una sola vez por cada nodo— de menor costo total [10]. En el resto del informe, la distancia se denota $d_{ij} = C_{ij}$.

Según cómo sean las distancias, el TSP puede ser simétrico o asimétrico (Tabla 1) [10].

Tabla 1: Diferencias entre TSP simétrico y asimétrico

	TSP Simétrico	**TSP Asimétrico**
Condición	$C_{ij} = C_{ji} \quad \forall i, j$	$C_{ij} \neq C_{ji}$ para algún i, j
Tipo de grafo	No dirigido	Dirigido
Ejemplo	Distancias en línea recta	Calles de un solo sentido

2.1. Formulación Matemática

La formulación clásica del TSP como problema de programación entera se debe a Dantzig, Fulkerson y Johnson [5]. Responde a tres preguntas: qué se decide, qué se minimiza y qué reglas debe cumplir la ruta.

Variables de decisión: qué se decide. Para cada par de ciudades se decide si se viaja directamente entre ellas: $x_{ij} = 1$ si el arco (o arista) (i, j) forma parte de la ruta, y $x_{ij} = 0$ si no.

Función objetivo: qué se minimiza. La suma de los costos de los arcos usados:

$$\text{mín} \sum_{i \in V} \sum_{j \in V,\, j \neq i} C_{ij}\, x_{ij}. \tag{1}$$

Restricciones de grado: se entra y se sale una vez. En el *TSP asimétrico*, cada ciudad tiene exactamente un arco de llegada y uno de salida:

$$\sum_{j \neq i} x_{ij} = 1 \quad \forall\, i \in V, \qquad \sum_{i \neq j} x_{ij} = 1 \quad \forall\, j \in V. \tag{2}$$

En el *TSP simétrico*, cada ciudad v toca exactamente dos aristas de la ruta, donde $\delta(v)$ es el conjunto de aristas que llegan a v:

$$\sum_{e \in \delta(v)} x_e = 2 \quad \forall\, v \in V. \tag{3}$$

Eliminación de subtours (DFJ): una sola ruta, no varias. Las restricciones de grado no bastan: una solución puede cumplirlas formando varios ciclos pequeños y separados (*subtours*) en vez de un único recorrido. Para impedirlo, dentro de cualquier grupo S de ciudades el número de arcos elegidos debe ser menor que $|S|$; así ningún grupo puede cerrarse sobre sí mismo:

$$\sum_{i \in S} \sum_{j \in S,\, j \neq i} x_{ij} \;\leq\; |S| - 1 \quad \forall\, S \subset V,\; 2 \leq |S| \leq n - 1. \tag{4}$$

2.2. Complejidad Computacional

Enunciar el TSP es fácil; resolverlo, no. El obstáculo es la cantidad de rutas posibles. Esta sección responde dos preguntas: *cuántas rutas hay* y *por qué el problema sigue siendo difícil* aunque no se revisen una por una.

¿Cuántas rutas distintas existen? Hay $n!$ formas de ordenar n ciudades, pero muchas describen el mismo recorrido:

- **El punto de partida no importa.** En un recorrido cerrado, $A \to B \to C \to D \to A$ y $B \to C \to D \to A \to B$ son la misma ruta. Cada ruta aparece n veces, una por cada ciudad desde la que se puede empezar.
- **El sentido no importa** (si $d_{ij} = d_{ji}$). Recorrer la ruta al revés da la misma distancia, así que cada ruta aparece 2 veces más.

Al dividir los $n!$ órdenes entre esas $n \cdot 2$ repeticiones quedan las rutas realmente distintas:

$$\frac{n!}{n \cdot 2} = \frac{(n-1)!}{2}, \qquad n \geq 3. \tag{5}$$

Por ejemplo, con cuatro ciudades los $4! = 24$ órdenes se reducen a solo $3!/2 = 3$ rutas: $ABCD$, $ABDC$ y $ACBD$. Todo otro orden es una rotación o una inversión de estas tres.

¿Qué tan rápido crece? La Tabla 2 evalúa (5) y la traduce a tiempo, suponiendo un computador muy rápido, capaz de evaluar 10^9 rutas por segundo.

Tabla 2: Número de rutas distintas según n y tiempo necesario para enumerarlas todas a razón de 10^9 rutas por segundo.

n	Rutas distintas $(n-1)!/2$	Tiempo de enumeración
5	12	< 1 μs
8	2 520	2,5 μs
10	181 440	0,18 ms
12	$2{,}00 \times 10^7$	20 ms
15	$4{,}36 \times 10^{10}$	44 s
20	$6{,}08 \times 10^{16}$	1,9 años
25	$3{,}10 \times 10^{23}$	$9{,}8 \times 10^6$ años
30	$4{,}42 \times 10^{30}$	$1{,}4 \times 10^{14}$ años

Para $n \leq 3$ existe una sola ruta; la fórmula (5) es válida desde $n = 3$.

La tabla deja dos conclusiones:

- **Cada ciudad nueva multiplica el trabajo por n.** Pasar de 15 a 20 ciudades multiplica el tiempo por más de un millón; con 30 ciudades, el cálculo tomaría unas diez mil veces la edad del universo.
- **Un computador más rápido no lo resuelve.** Uno mil veces más veloz apenas permite pasar de 20 a 22 ciudades: ese salto ya multiplica el trabajo por $20 \cdot 21 = 420$, y llegar a 23 lo multiplicaría por más de 9 000.

El problema es el TSP, no la forma de buscar. Podría pensarse que basta con una estrategia más astuta que listar rutas. La teoría de la complejidad indica que no. Para verlo se usan dos clases de problemas de decisión, los que se responden con «sí» o «no»:

- **P:** problemas que se *resuelven* en tiempo polinomial.
- **NP:** problemas cuya respuesta afirmativa se puede *verificar* en tiempo polinomial [8].

Es la diferencia de un sudoku: llenarlo puede costar mucho, pero revisar uno ya lleno toma segundos.

La *versión de decisión* del TSP pregunta: ¿existe una ruta de longitud a lo sumo k? Está en NP, porque basta sumar las n aristas de una ruta y comparar con k. Además es **NP-completa**. Karp [14] demostró que decidir si un grafo tiene un ciclo hamiltoniano es NP-completo, y ese problema se convierte en un TSP asignando

$$d_{ij} = \begin{cases} 1, & \text{si } \{i, j\} \text{ es arista del grafo,} \\ 2, & \text{en caso contrario.} \end{cases} \tag{6}$$

Toda ruta usa n aristas. Con estos costos mide exactamente n solo si todas sus aristas cuestan 1, es decir, si todas existían en el grafo original. Por eso, *existe una ruta de longitud a lo sumo n si y solo si el grafo tiene un ciclo hamiltoniano.* Un método rápido para el TSP resolvería también ese problema y, con él, todos los de NP.

Como encontrar la ruta *más corta* responde de inmediato la pregunta de decisión, la versión de optimización del TSP es **NP-difícil** [8]: un algoritmo polinomial para ella implicaría $P = NP$, algo que casi nadie cree cierto.

Matiz y cierre. NP-difícil habla del *peor caso*, no de cada instancia. Hay métodos exactos que han resuelto instancias de decenas de miles de ciudades [1], aunque su esfuerzo sigue creciendo de forma exponencial en el peor caso. Por eso hay que elegir: exigir el óptimo y aceptar un costo que se dispara con n, o renunciar a esa garantía a cambio de tiempos manejables.

3. Métodos de Resolución del TSP

La Sección 2.2 mostró que no se conoce (y probablemente no exista) un algoritmo que encuentre el óptimo del TSP en tiempo polinomial. Por eso, todo método debe elegir entre dos objetivos que no se pueden cumplir a la vez:

garantía de optimalidad frente a *costo computacional acotado.*

Cada familia de métodos toma una decisión distinta:

- **Métodos exactos** (Sección 3.1): garantizan la ruta óptima, pero su tiempo crece de forma exponencial y dejan de ser viables con pocas decenas de ciudades.
- **Métodos heurísticos** (Sección 3.2): dan una buena ruta en tiempo polinomial, sin garantizar que sea la mejor.

- **Metaheurísticas** (Sección 3.3): corrigen la principal debilidad de las heurísticas, quedarse atascadas.

Todos comparan rutas con el mismo criterio: su longitud total. Si π es una permutación de las n ciudades que representa un recorrido cerrado, su costo es

$$L(\pi) \;=\; \sum_{i=1}^{n-1} d_{\pi_i \pi_{i+1}} \;+\; d_{\pi_n \pi_1}, \tag{7}$$

y el problema consiste en minimizar L sobre el conjunto Π de todas las permutaciones. Esta función se analiza en detalle en la Sección 5.2; se presenta aquí porque es la que evalúan todos los algoritmos siguientes.

3.1. Métodos Exactos

Un método exacto entrega una ruta cuya optimalidad está demostrada [1]. Como hay demasiadas rutas (Sección 2.2), la única forma de lograrlo en un tiempo razonable es *descartar* la mayoría sin evaluarlas. Los tres métodos siguientes se diferencian justamente en cómo lo hacen: la fuerza bruta no descarta nada, la programación dinámica evita repetir cálculos y el *branch and bound* poda ramas completas usando cotas.

3.1.1. Fuerza Bruta

Revisa los $(n-1)!/2$ recorridos distintos, calcula la longitud (7) de cada uno y se queda con el menor. Es aplicar literalmente la definición del problema:

$$\pi^* \;=\; \arg\min_{\pi \in \Pi} L(\pi). \tag{8}$$

Algoritmo 1 Fuerza bruta aplicada al TSP

```
L* ← +∞
for all π ∈ Π (las (n − 1)!/2 rutas distintas) do
    Calcular L(π) con (7)
    if L(π) < L* then
        L* ← L(π);    π* ← π
    end if
end for
return π*
```

Siempre acierta, porque revisa todo. Pero cuesta $O(n!)$ operaciones: hay del orden de $n!$ recorridos y evaluar cada uno cuesta $O(n)$. Según la Tabla 2, incluso a 10^9 rutas por segundo deja de ser práctica pasadas las 15 ciudades: con 17 tomaría unas tres horas y con 20, casi dos años. Su uso es didáctico y de verificación: comprueba en casos pequeños lo que entregan métodos más sofisticados.

Idea clave. Revisa todas las rutas y elige la más corta. Siempre acierta, pero solo es viable con muy pocas ciudades.

Analogía. Como abrir un candado probando todas las combinaciones: tarde o temprano se abre, pero con muchos dígitos no alcanza el tiempo.

3.1.2. Programación Dinámica de Held–Karp

Bellman [2] y, por separado, Held y Karp [12] notaron que la fuerza bruta repite trabajo. Se fija la ciudad 1 como origen y se usa esta idea:

> *El mejor camino que llega a la ciudad j habiendo visitado un conjunto S de ciudades no depende del orden en que se recorrió S, sino solo de qué ciudades contiene S y de cuál fue la última.*

Es el principio de optimalidad de Bellman, y permite agrupar muchos órdenes en un solo *estado*. Por ejemplo, $1 \to 2 \to 3 \to 5$ y $1 \to 3 \to 2 \to 5$ terminan en el mismo estado ($S = \{2, 3, 5\}$, $j = 5$): basta guardar el más corto y olvidar el otro.

Formalmente, para $S \subseteq V \setminus \{1\}$ y $j \in S$, sea $C(S, j)$ la longitud del camino más corto que sale de la ciudad 1, visita exactamente las ciudades de S y termina en j. Entonces

$$C(\{j\},\, j) = d_{1j}, \tag{9}$$

$$C(S, j) = \min_{i \in S \setminus \{j\}} \Big[\, C\big(S \setminus \{j\},\, i\big) + d_{ij} \,\Big], \qquad |S| \geq 2, \tag{10}$$

y la ruta óptima se obtiene cerrando el ciclo desde la última ciudad:

$$L^{*} \;=\; \min_{j \in V \setminus \{1\}} \Big[\, C\big(V \setminus \{1\},\, j\big) + d_{j1} \,\Big]. \tag{11}$$

La ecuación (10) dice que, para llegar a j pasando por S, primero hubo que llegar a otra ciudad i de S y luego dar el paso $i \to j$. Cada estado depende de estados con un elemento menos, así que se calculan de menor a mayor tamaño de S.

Algoritmo 2 Programación dinámica de Held–Karp

for all $j \in V \setminus \{1\}$ **do**
  $C(\{j\}, j) \leftarrow d_{1j}$ ▷ caso base (9)
end for
for $s = 2$ **hasta** $n - 1$ **do**
  for all $S \subseteq V \setminus \{1\}$ con $|S| = s$ **do**
    for all $j \in S$ **do**
      $C(S, j) \leftarrow \min_{i \in S \setminus \{j\}} \big[C(S \setminus \{j\}, i) + d_{ij}\big]$
      $p(S, j) \leftarrow$ el i que alcanzó el mínimo ▷ para reconstruir
    end for
  end for
end for
$L^{*} \leftarrow \min_{j} \big[C(V \setminus \{1\}, j) + d_{j1}\big]$ ▷ cierre (11)
Reconstruir π^{*} hacia atrás usando p
return π^{*}

El costo sale de contar estados: hay $(n-1)\,2^{\,n-2}$ pares (S, j) y cada uno requiere a lo sumo $n-2$ comparaciones, de modo que

$$\text{tiempo } O\big(n^{2} 2^{n}\big), \qquad \text{memoria } O(n\, 2^{n}). \tag{12}$$

Para $n = 20$ son unas 4×10^8 operaciones, menos de un segundo, frente a los 6×10^{16} recorridos de la fuerza bruta. Sigue siendo exponencial, y su límite real es la memoria: con $n = 25$ hay que guardar unos 2×10^8 valores (casi 1 GB a 4 bytes cada uno, y el doble si también se guarda la tabla p para reconstruir la ruta); con $n = 30$, unos 8×10^9, más de 30 GB. En la práctica se agota la memoria antes que la paciencia, en torno a las 25 o 30 ciudades.

Idea clave. Guarda el mejor camino para cada «estado» (ciudades ya visitadas y última ciudad) y lo reutiliza en lugar de recalcularlo.

Analogía. Como anotar en una libreta el mejor camino hacia cada punto intermedio para no volver a calcularlo: ahorra mucho trabajo, pero la libreta crece hasta volverse inmanejable.

3.1.3. Ramificación y Acotamiento (*Branch and Bound*)

Propuesto por Land y Doig [16] y aplicado al TSP por Little et al. [18], organiza las rutas como un *árbol de decisiones* y descarta ramas completas sin recorrerlas: una rama se abandona apenas lo ya recorrido, más lo mínimo que falta, supera a la mejor ruta conocida. Funciona en tres pasos.

Ramificar. Cada nodo del árbol es un subproblema con algunas decisiones ya tomadas. Usando la formulación de la Sección 2.1, un nodo se divide en dos: en uno la ruta debe usar la arista (i, j), es decir $x_{ij} = 1$, y en el otro no puede usarla, $x_{ij} = 0$.

Acotar. Para estimar «lo mínimo que falta» se resuelve en cada nodo una *relajación*: el mismo problema sin las restricciones difíciles, por ejemplo permitiendo $0 \le x_{ij} \le 1$ en lugar de $x_{ij} \in \{0, 1\}$. Como la relajación admite más soluciones, su valor $LB(\nu)$ nunca supera al de ninguna ruta del nodo ν:

$$LB(\nu) \;\le\; \text{mín}\{\, L(\pi) \;:\; \pi \text{ compatible con las decisiones de } \nu \,\}. \tag{13}$$

Podar. Sea U la longitud de la mejor ruta encontrada hasta el momento (cota superior). Si

$$LB(\nu) \;\ge\; U, \tag{14}$$

ninguna ruta de esa rama puede mejorar a la actual, y la rama se descarta completa. También se descarta si la relajación no tiene solución.

Algoritmo 3 Ramificación y acotamiento aplicado al TSP

Obtener una ruta inicial π^* con una heurística (p. ej. vecino más cercano)
$U \leftarrow L(\pi^*)$ ▷ cota superior inicial
$Q \leftarrow \{$nodo raíz, sin decisiones fijadas$\}$
while $Q \neq \emptyset$ **do**
    Extraer un nodo ν de Q y calcular $LB(\nu)$
    if $LB(\nu) \geq U$ **o** la relajación es infactible **then**
        Descartar ν ▷ poda, criterio (14)
    else if la solución de la relajación ya es un recorrido completo **then**
        $U \leftarrow LB(\nu)$; $\pi^* \leftarrow$ ese recorrido
    else
        Ramificar ν en dos subproblemas ($x_{ij} = 1$ y $x_{ij} = 0$) y añadirlos a Q
    end if
end while
return π^*

El rendimiento depende de la calidad de las cotas. Con cotas ajustadas se poda pronto y se visita una fracción mínima del árbol; con cotas flojas hay que recorrerlo casi entero y el método se vuelve fuerza bruta. Por eso ayuda partir de una buena ruta inicial: una U baja hace que (14) se cumpla más veces. Es el primer vínculo entre los métodos exactos y los heurísticos. En el peor caso, el costo sigue siendo exponencial.

Solvers modernos. La variante más potente, *branch and cut*, agrega las restricciones de subtours de Dantzig, Fulkerson y Johnson [5] solo cuando la relajación las viola, en vez de incluirlas todas desde el inicio (son exponencialmente muchas). Es la base de Concorde, el programa que resolvió instancias de decenas de miles de ciudades [1]. Implementarlo exige gestionar los nodos abiertos, resolver una relajación por nodo y manejar las restricciones añadidas; de ahí su baja facilidad de implementación en la Sección 4.

Idea clave. Explora las rutas como un árbol y descarta ramas enteras cuando su cota inferior ya no puede mejorar a la mejor ruta conocida.

Analogía. Como buscar el pasaje más barato: si una escala ya cuesta más que el mejor precio encontrado, se descartan todos los vuelos que pasan por ella sin revisarlos.

3.2. Métodos Heurísticos

Cuando la garantía de optimalidad cuesta demasiado, se renuncia a ella. Una **heurística** es una regla práctica que construye o mejora una ruta en tiempo polinomial, sin asegurar que sea la óptima [6]. Hay dos tipos:

- **Constructivas:** arman una ruta desde cero, ciudad por ciudad (vecino más cercano, inserción más barata).
- **De mejora:** parten de una ruta completa y la modifican mientras encuentren cambios que la acorten (2-opt).

En la práctica se combinan: una constructiva arma la ruta inicial y una de mejora la refina. Para medir qué tan buena es la ruta obtenida se usa el *gap* relativo de la Sección 5.2.

3.2.1. Vecino Más Cercano

Es la regla más natural: desde la ciudad actual, ir siempre a la ciudad no visitada más cercana y, al final, volver al origen. Si R es el conjunto de ciudades pendientes, el paso k elige

$$\pi_k \;=\; \arg\min_{j \in R} \; d_{\pi_{k-1}\, j}. \tag{15}$$

Algoritmo 4 Vecino más cercano aplicado al TSP

1: Elegir una ciudad inicial π_1; $\quad R \leftarrow V \setminus \{\pi_1\}$
2: **for** $k = 2$ **hasta** n **do**
3: $\quad \pi_k \leftarrow \arg\min_{j \in R} d_{\pi_{k-1} j}$ ▷ regla (15)
4: $\quad R \leftarrow R \setminus \{\pi_k\}$
5: **end for**
6: **return** $\pi = (\pi_1, \ldots, \pi_n)$, cerrando el ciclo con la arista (π_n, π_1)

Cada uno de los $n-1$ pasos revisa a lo sumo n ciudades, así que cuesta $O(n^2)$. Su debilidad es la *miopía*: las primeras decisiones son buenas, pero al final quedan ciudades dispersas que obligan a usar aristas largas, incluida la de regreso. Rosenkrantz, Stearns y Lewis [24] demostraron que, en el peor caso y aun con distancias métricas, la ruta puede ser peor que la óptima por un factor que crece con $\log n$. En instancias típicas queda alrededor de un 25 % por encima del óptimo [13].

Idea clave. Desde cada ciudad, ir a la más cercana que falte. Es rápido, pero las últimas conexiones suelen ser largas.

Analogía. Como hacer compras yendo siempre a la tienda más cercana: el inicio es eficiente, pero al final queda un viaje largo hasta la tienda que se dejó para el último.

3.2.2. Inserción Más Barata

En lugar de avanzar desde un extremo, mantiene un *ciclo cerrado* que va creciendo. Se parte de un ciclo con dos ciudades y, en cada paso, se inserta la ciudad que menos alarga el recorrido. Insertar la ciudad k entre dos ciudades consecutivas i y j quita la arista (i, j) y agrega (i, k) y (k, j); el aumento de longitud es

$$\Delta_{ijk} \;=\; d_{ik} + d_{kj} - d_{ij}, \tag{16}$$

es decir, lo que se agrega menos lo que se quita. En cada paso se elige la combinación que minimiza (16) entre todas las aristas del ciclo y todas las ciudades pendientes.

Algoritmo 5 Inserción más barata aplicada al TSP

1: Formar un ciclo inicial C con dos ciudades; $\quad R \leftarrow V \setminus C$
2: **while** $R \neq \emptyset$ **do**
3: $\quad (i^*, j^*, k^*) \leftarrow \arg\min\{\, \Delta_{ijk} \;:\; (i,j) \in C,\; k \in R \,\}$
4: $\quad$ Insertar k^* entre i^* y j^* en C
5: $\quad R \leftarrow R \setminus \{k^*\}$
6: **end while**
7: **return** C

A diferencia del vecino más cercano, cada decisión mira la ruta completa y no solo el último tramo. Por eso suele dar mejores rutas y, si las distancias cumplen la desigualdad triangular, su longitud nunca supera el doble de la óptima [24], una garantía que el vecino más cercano no tiene. A cambio, cuesta más: $O(n^3)$ en su versión directa, reducible a $O(n^2 \log n)$ con estructuras auxiliares [24].

Idea clave. Construye un ciclo e inserta cada ciudad donde menos alarga el recorrido.

Analogía. Como agregar paradas a una ruta de reparto ya planificada: cada nueva parada se coloca donde genera el menor desvío.

3.2.3. Intercambio 2-opt

Es una heurística de *mejora* propuesta por Croes [4]. Toma dos aristas no adyacentes (a, b) y (c, e) —con b antes que c en el recorrido—, las elimina y reconecta los extremos de la única otra forma que mantiene un solo ciclo: agrega (a, c) y (b, e). Esto equivale a invertir el tramo entre b y c:

$$\ldots, a, \underbrace{b, \ldots, c}_{\text{se invierte}}, e, \ldots \;\longrightarrow\; \ldots, a, \underbrace{c, \ldots, b}_{\text{invertido}}, e, \ldots$$

Las demás aristas no cambian, así que la variación de longitud depende solo de cuatro distancias:

$$\Delta \;=\; \big(d_{ac} + d_{be}\big) - \big(d_{ab} + d_{ce}\big). \tag{17}$$

Si $\Delta < 0$, el cambio conviene. Calcular (17) cuesta $O(1)$, un esfuerzo constante sin importar n; esta *evaluación incremental* se detalla en la Sección 5.2 y es la que permite hacer millones de evaluaciones.

Algoritmo 6 2-opt aplicado al TSP

```
Partir de una ruta π (p. ej. construida con vecino más cercano)
repeat
    mejora ← falso
    for all pares de aristas no adyacentes (a, b) y (c, e) de π do
        Δ ← (d_ac + d_be) − (d_ab + d_ce)                    ▷ ecuación (17)
        if Δ < 0 then
            Invertir en π el tramo de b a c;    mejora ← verdadero
        end if
    end for
until mejora = falso
return π
```

Con distancias euclidianas tiene una lectura geométrica sencilla: dos aristas que se cruzan siempre pueden reemplazarse por dos más cortas, así que 2-opt equivale a *eliminar los cruces* de la ruta. Cada pasada revisa $O(n^2)$ pares de aristas, y el algoritmo se detiene cuando ningún intercambio acorta la ruta. Partiendo de una ruta razonable, suele quedar a pocos puntos porcentuales del óptimo [13].

Idea clave. Toma una ruta completa y elimina sus cruces invirtiendo tramos, mientras eso la acorte.

Analogía. Como desenredar un cable: se deshace cada cruce hasta que queda ordenado y más corto.

El límite común de las tres heurísticas. Las tres toman siempre la decisión que más conviene en el momento, sin retroceder ni empeorar la ruta a propósito. Ahí está su límite: el vecino más cercano y la inserción nunca revisan una decisión tomada, y 2-opt se detiene cuando ningún intercambio individual acorta la ruta, aunque exista otra mucho mejor fuera de su alcance. Esa solución es un **óptimo local**: la mejor de su entorno inmediato, no la mejor de todas. Es como un excursionista que busca el punto más bajo caminando siempre cuesta abajo: termina en el primer valle que encuentra, sin ver el más profundo que hay detrás de la colina.

Salir de ahí exige aceptar, en algún momento, soluciones peores. Esa es la idea de las metaheurísticas (Sección 3.3). La comparación completa entre métodos exactos y aproximados se desarrolla en la Sección 4.

3.3. Metaheurísticas

Las heurísticas de la Sección 3.2 aplican una regla fija, pensada para el TSP, y se detienen en el primer óptimo local que encuentran. Una **metaheurística** trabaja un nivel por encima: es una estrategia general de búsqueda, aplicable a muchos problemas, que dirige a heurísticas subordinadas —por ejemplo, al propio 2-opt— y les permite escapar de los óptimos locales aceptando, de forma controlada, soluciones peores [19, 3]. Siguiendo la analogía anterior, el excursionista acepta subir la cuesta para llegar al otro valle. La diferencia entre ambas es de papel: la heurística decide *qué movimiento hacer*; la metaheurística decide *cuándo aceptar un movimiento que empeora* y hacia dónde seguir buscando. La Tabla 3 resume el contraste.

Tabla 3: Diferencias entre heurísticas y metaheurísticas

Aspecto	**Heurística**	**Metaheurística**
Alcance	Regla diseñada para un problema concreto.	Estrategia general, adaptable a muchos problemas.
Movimientos aceptados	Solo los que mejoran la ruta.	También algunos que la empeoran, de forma controlada.
Resultado típico	Se detiene en un óptimo local.	Escapa de óptimos locales, aunque sin garantía de llegar al global.
Ejemplos	Vecino más cercano, inserción más barata, 2-opt.	Recocido simulado, algoritmos genéticos, colonia de hormigas.

Todas las metaheurísticas se organizan en torno a dos fuerzas opuestas: la **intensificación**, que refina lo ya encontrado, y la **diversificación**, que explora regiones nuevas. Su equilibrio define el comportamiento del algoritmo y es el criterio que ordena la Sección 5.1.

Se agrupan según cuántas soluciones manejan a la vez [3]. Los **métodos de trayectoria** modifican una sola ruta paso a paso (recocido simulado); los **poblacionales** mantienen un conjunto de rutas que compiten o cooperan (algoritmos genéticos, colonia de hormigas).

3.3.1. Recocido Simulado (*Simulated Annealing*)

En metalurgia, el recocido calienta un metal y lo enfría lentamente: a temperatura alta los átomos prueban muchas configuraciones, y al bajar quedan fijados en una de energía mínima. Un enfriamiento brusco lo deja con defectos, atrapado en una configuración peor.

Kirkpatrick, Gelatt y Vecchi [15] llevaron ese proceso a la optimización: la energía equivale al costo de la ruta y la temperatura, a la disposición del algoritmo a aceptar soluciones peores. El método empieza «caliente», probando casi cualquier cambio, y termina «frío», admitiendo solo mejoras.

Dada una ruta π y una vecina π', sea $\Delta = L(\pi') - L(\pi)$ la variación de costo. La vecina se acepta con probabilidad

$$P(\text{aceptar } \pi') = \begin{cases} 1 & \text{si } \Delta \leq 0, \\ \exp\left(-\dfrac{\Delta}{T}\right) & \text{si } \Delta > 0, \end{cases} \tag{18}$$

donde T es la temperatura actual. Es el criterio de Metropolis, y se lee así:

- Si el cambio mejora la ruta ($\Delta \leq 0$), se acepta siempre.
- Con T grande, la probabilidad se acerca a uno: casi todo empeoramiento se acepta y el método *diversifica*.
- Con T pequeña, la probabilidad tiende a cero: solo pasan las mejoras y el método *intensifica*.
- A una misma temperatura, los empeoramientos grandes se rechazan más que los pequeños, así que nunca se destruye una buena ruta de golpe.

La ruta vecina se genera con un movimiento **2-opt** [4]: se eliminan dos aristas y se reconectan los extremos al revés, lo que invierte un tramo del recorrido. Es un cambio pequeño y barato de calcular.

El descenso de T sigue un **esquema de enfriamiento**; el más usado es el geométrico [6, 26]:

$$T_{k+1} = \alpha\, T_k, \qquad 0 < \alpha < 1. \tag{19}$$

Esta ecuación es el motor del método: lleva al algoritmo, poco a poco, del modo exploratorio al conservador.

Algoritmo 7 Recocido simulado aplicado al TSP

Generar una ruta inicial π (p. ej. con vecino más cercano)
$T \leftarrow T_0$; $\quad \pi^* \leftarrow \pi$
while $T > T_{\text{mín}}$ **do**
  for $k = 1$ **hasta** L_{iter} **do**
    Generar un vecino π' mediante un movimiento 2-opt
    $\Delta \leftarrow L(\pi') - L(\pi)$
    if $\Delta \leq 0$ **o** $\text{rand}(0,1) < \exp(-\Delta/T)$ **then**
      $\pi \leftarrow \pi'$
    end if
    if $L(\pi) < L(\pi^*)$ **then**
      $\pi^* \leftarrow \pi$
    end if
  end for
  $T \leftarrow \alpha\, T$
end while
return π^*

Sus parámetros son T_0 (temperatura inicial), α (tasa de enfriamiento), $T_{\text{mín}}$ (temperatura de corte) y L_{iter} (iteraciones por nivel), analizados en la Sección 5.1.

Idea clave. Mejora una sola ruta y acepta a veces cambios que la empeoran; esa tolerancia baja a medida que el algoritmo «se enfría».

Analogía. Como buscar las llaves en una casa: al principio se revisan cuartos al azar, incluso los poco probables; con el tiempo, la búsqueda se concentra solo en los lugares más prometedores.

3.3.2. Algoritmos Genéticos

Se inspiran en la selección natural [9]. En lugar de una sola ruta, mantienen una **población** de rutas que evoluciona por **generaciones**. Las rutas cortas tienen más probabilidad de ser elegidas como padres, y sobre sus hijos actúan dos operadores: el **cruce**, que combina fragmentos de dos padres, y la **mutación**, que introduce cambios al azar. Tras muchas generaciones, la población llega a buenas soluciones sin que ningún paso haya calculado la ruta óptima.

Cada ruta se representa con la **representación de ruta**: el cromosoma es la propia permutación π [17]. Por ejemplo, el individuo $(3, 1, 5, 2, 4)$ visita la ciudad 3, luego la 1, y así hasta volver al origen.

Cada individuo se evalúa con una **función de aptitud**. Aquí hay un detalle: el TSP *minimiza* distancia, pero la selección favorece la aptitud *máxima*. Se resuelve invirtiendo el costo:

$$f(\pi) = \frac{1}{L(\pi)}. \tag{20}$$

Así, la ruta más corta pasa a ser la de mayor aptitud.

Los padres se eligen por **torneo**: se toman al azar unos pocos individuos y se conserva el de menor costo. Premia la calidad sin eliminar del todo a los mediocres, lo que mantiene la diversidad.

El cruce merece atención aparte, porque el cruce clásico de un punto **no funciona en el TSP**:

Padre 1	1	2	3	4	5	6
Padre 2	3	1	5	6	2	4
Hijo	1	2	3	6	2	4

En el hijo, la ciudad 2 aparece dos veces y la 5 desaparece: no es una ruta válida. El **cruce OX** (*Order Crossover*) lo evita [17]: copia un tramo del primer padre y completa el resto con las ciudades que faltan, en el orden en que aparecen en el segundo. El hijo hereda un tramo de uno y el orden de visita del otro, y siempre es una ruta válida.

La **mutación** aporta variedad que el cruce no puede generar, porque este solo recombina lo que ya existe; se aplica intercambiando dos ciudades o invirtiendo un tramo [17]. El **elitismo** pasa intactos a los mejores individuos a la siguiente generación, para que la mejor ruta no se pierda por azar.

Algoritmo 8 Algoritmo genético aplicado al TSP

Inicializar una población P de N rutas
Evaluar $f(\pi)$ para toda $\pi \in P$
for $g = 1$ **hasta** $G_{\text{máx}}$ **do**
    $P' \leftarrow$ los e mejores individuos de P ▷ elitismo
    while $|P'| < N$ **do**
        Seleccionar dos padres π_1, π_2 de P
        Aplicar cruce OX con probabilidad p_c y obtener π_h
        Aplicar mutación a π_h con probabilidad p_m
        $P' \leftarrow P' \cup \{\pi_h\}$
    end while
    $P \leftarrow P'$; evaluar f sobre P
end for
return la mejor ruta encontrada

Sus parámetros son N (población), p_c y p_m (probabilidades de cruce y mutación), $G_{\text{máx}}$ (generaciones) y e (élite).

Idea clave. Mantiene muchas rutas a la vez: combina las mejores (cruce), introduce pequeños cambios (mutación) y protege a las mejores (elitismo).

Analogía. Como la crianza selectiva de plantas: se cruzan los mejores ejemplares y, generación tras generación, la cosecha mejora.

3.3.3. Optimización por Colonia de Hormigas (ACO)

Las hormigas dejan feromona al caminar. Las que toman el camino más corto lo recorren más veces en el mismo tiempo, acumulan más feromona y atraen a las siguientes. Así, la ruta corta surge del comportamiento de la colonia sin que ninguna hormiga la calcule.

Dorigo, Maniezzo y Colorni [7] llevaron esta idea al algoritmo *Ant System*. Hormigas artificiales construyen rutas combinando dos fuentes de información: la feromona acumulada (experiencia de la colonia) y la distancia a la ciudad candidata (información local). Una hormiga en la ciudad i elige la ciudad j con probabilidad

$$p_{ij} = \frac{[\tau_{ij}]^{\alpha}\,[\eta_{ij}]^{\beta}}{\sum_{l \in N_i} [\tau_{il}]^{\alpha}\,[\eta_{il}]^{\beta}}, \qquad j \in N_i, \tag{21}$$

donde τ_{ij} es la feromona en la arista (i, j), $\eta_{ij} = 1/d_{ij}$ es la **visibilidad** (mayor cuanto más cerca) y N_i son las ciudades aún no visitadas.

El numerador multiplica ambas fuentes y los exponentes fijan su peso: α alto hace que la hormiga confíe en la memoria de la colonia; β alto la vuelve avara, atenta solo a lo cercano. El denominador hace que las probabilidades sumen uno. (Este α no es la tasa de enfriamiento del recocido simulado; solo comparten el símbolo.)

Al final de cada iteración, la feromona se actualiza:

$$\tau_{ij} \leftarrow (1-\rho)\,\tau_{ij} + \sum_{k=1}^{m} \Delta\tau_{ij}^{k}, \qquad \Delta\tau_{ij}^{k} = \begin{cases} \dfrac{Q}{L(\pi_k)} & \text{si la hormiga } k \text{ usó } (i,j), \\ 0 & \text{en caso contrario,} \end{cases} \tag{22}$$

con $\rho \in (0,1)$ la tasa de evaporación, m el número de hormigas y Q una constante. La ecuación tiene dos partes:

- **Depósito** (segundo término): cada hormiga deja Q dividido entre la longitud de su ruta, así que las rutas cortas refuerzan más sus aristas. Ahí está el aprendizaje.
- **Evaporación** (primer término): borra una parte de la feromona en cada iteración. Sin ella, las primeras rutas —hechas casi al azar— dominarían la búsqueda y la colonia se estancaría antes de tiempo [7].

Algoritmo 9 *Ant System* aplicado al TSP

```
Inicializar τ_ij ← τ_0 para toda arista (i, j)
for t = 1 hasta t_máx do
    for cada hormiga k = 1, ..., m do
        Colocar la hormiga en una ciudad inicial
        Construir una ruta π_k aplicando la regla (21)
        Evaluar L(π_k)
    end for
    Actualizar la feromona según (22)
end for
return la mejor ruta encontrada
```

Sus parámetros son m (hormigas), α y β (pesos), ρ (evaporación), Q (depósito) y $t_{\text{máx}}$ (iteraciones).

Idea clave. Cada hormiga arma una ruta eligiendo el siguiente punto según la distancia y la feromona; las rutas cortas refuerzan sus aristas y la evaporación borra lo que no sirve.

Analogía. Como los senderos que se forman en el pasto de un parque: el camino que más gente usa se marca y atrae a más personas, mientras los que nadie pisa desaparecen con el tiempo.

Tabla 4: Comparación de las tres metaheurísticas aplicadas al TSP

Método	**Tipo de búsqueda**	**Mecanismo de escape del óptimo local**	**Parámetros principales**
Recocido simulado	Trayectoria (una sola solución)	Aceptación probabilística de empeoramientos, controlada por la temperatura	T_0, α, $T_{\text{mín}}$, L_{iter}
Algoritmo genético	Poblacional	Mutación aleatoria y diversidad genética de la población	N, p_c, p_m, $G_{\text{máx}}$, e
Colonia de hormigas	Poblacional constructiva	Evaporación de feromona y construcción estocástica de la ruta	m, α, β, ρ, Q, $t_{\text{máx}}$

4. Comparativa Técnica: Métodos Exactos vs. Métodos Heurísticos

La Sección 3 presentó tres familias de métodos. La Tabla 5 las compara según los cuatro criterios que deciden cuál conviene usar. Las heurísticas y las metaheurísticas comparten una columna porque ambas renuncian a la garantía de optimalidad a cambio de tiempos manejables.

Tabla 5: Comparativa entre métodos exactos y métodos heurísticos/metaheurísticos

Característica	**Métodos Exactos**	**Métodos Heurísticos / Metaheurísticos**
Garantía de optimalidad	Sí. La solución devuelta es demostrablemente la mejor.	No. Buena solución, sin certeza de que sea la óptima.
Escalabilidad (n.º de ciudades)	Muy baja. El esfuerzo crece de forma factorial o exponencial con n.	Alta. Resuelven miles de ciudades en tiempos razonables.
Costo computacional	Muy alto y no controlable: lo impone el tamaño de la instancia.	Moderado y ajustable: el usuario fija cuánto tiempo invertir.
Facilidad de implementación	Baja. Requiere estructuras de poda y manejo de memoria.	Media. Estructura simple, pero exige calibrar parámetros.

La tabla resume la disyuntiva planteada en la Sección 2.2. Los métodos exactos convienen cuando la instancia es pequeña —hasta unas 25 ciudades con Held–Karp— o cuando es indispensable demostrar el óptimo; solvers como Concorde amplían ese límite, pero a un costo computacional y de implementación muy alto [1]. Las heurísticas y metaheurísticas convienen cuando la instancia es grande o el tiempo es limitado. En la práctica se combinan: una heurística aporta la ruta inicial, una metaheurística la mejora y, si el tamaño lo permite, un método exacto certifica el resultado.

5. El Factor de Optimización: Parámetros y Función de Costo

5.1. Parámetros a Ajustar

Una metaheurística no es un procedimiento cerrado, sino una plantilla que hay que configurar; mal calibrada, puede rendir peor que una heurística simple. Al revisar los tres métodos aparece un patrón: cada parámetro, sea del método que sea, regula la misma tensión entre intensificación y diversificación [3, 19]. Fijar su valor es decidir en qué punto de ese eje trabajará el algoritmo. Por eso la Tabla 6 describe cada parámetro por lo que pasa en sus extremos y no por su definición.

Tabla 6: Parámetros principales de cada metaheurística y su efecto sobre la búsqueda

Parám.	**Qué controla**	**Si es muy alto**	**Si es muy bajo**
Recocido simulado			
T_0	Exploración inicial	Acepta casi todo durante muchas iteraciones: busca al azar y desperdicia cómputo.	Arranca «frío» y solo admite mejoras: se atasca en el primer óptimo local.
α	Velocidad de enfriamiento	Enfría muy lento: explora a fondo, pero el tiempo se dispara.	Enfría de golpe: se congela sin haber explorado y devuelve una ruta mediocre.
L_{iter}	Iteraciones por nivel de T	Aprovecha cada temperatura: más calidad a costa de mucho tiempo.	No deja estabilizarse en cada nivel y el enfriamiento gradual pierde sentido.
Algoritmo genético			
N	Diversidad genética	Mucha variedad, pero cada generación cuesta más y la convergencia es lenta.	La población se uniformiza rápido: convergencia prematura.
p_c	Frecuencia de recombinación	Destruye combinaciones buenas antes de que se consoliden.	Casi no genera descendientes nuevos: la población no evoluciona.
p_m	Variación nueva	Degenera en búsqueda aleatoria y pierde lo aprendido.	No entra información nueva: la población queda atrapada.
e	Preservación de los mejores	Unos pocos individuos dominan y la diversidad desaparece.	La mejor ruta puede perderse de una generación a otra.
Colonia de hormigas			
α	Peso de la feromona	La colonia sigue ciegamente el rastro y todas repiten la misma ruta: estancamiento.	Se ignora la feromona: no hay aprendizaje, queda una heurística avara.
β	Peso de la distancia	Prima siempre la ciudad más cercana: se aproxima al vecino más cercano.	Se desatiende la distancia: las primeras rutas son malas y el aprendizaje arranca lentísimo.
ρ	Olvido de lo aprendido	Borra la información demasiado rápido y nunca consolida buenas rutas.	La feromona se acumula sin límite: convergencia prematura.
m	Exploración por iteración	Más rutas evaluadas, pero el costo por iteración crece igual.	Actualiza la feromona con evidencia escasa y poco fiable.

Las once filas describen mecanismos distintos, pero sus columnas extremas muestran siempre el mismo par de fallos: explorar tanto que no se converge, o converger tan pronto que no se explora. No es casualidad: es la estructura común a todas las metaheurísticas [3]. Por eso no existe una configuración óptima para todos los casos; depende de la instancia y del tiempo disponible [6, 26]. En la práctica, los valores se fijan por experimentación manual, búsqueda sobre una malla de combinaciones (*grid search*) o procedimientos automáticos de configuración [26].

5.2. Función Objetivo / Función de Costo

Todos los métodos anteriores usan el mismo criterio para decidir si una ruta es mejor que otra. Ese criterio es la función objetivo, y es lo que convierte al TSP en un problema de optimización.

Si $\pi = (\pi_1, \pi_2, \ldots, \pi_n)$ es una permutación de las n ciudades de V que representa un recorrido

cerrado, su costo $L(\pi)$ es el de la ecuación (7), y el problema consiste en resolver

$$\min_{\pi \in \Pi} \; L(\pi), \tag{23}$$

donde Π es el conjunto de todas las permutaciones de V. La sumatoria recorre los tramos consecutivos, y el término final $d_{\pi_n \pi_1}$ es el regreso al origen: el que cierra el ciclo. Sin él se trataría de un problema de ruta abierta, distinto y más sencillo.

Evaluación incremental del costo

Recalcular la ecuación (7) completa en cada evaluación cuesta n sumas. Pero un movimiento 2-opt cambia solo dos aristas: elimina (a, b) y (c, e), e incorpora (a, c) y (b, e). Las demás se cancelan al restar, así que basta evaluar la diferencia Δ de la ecuación (17), que solo usa cuatro distancias. Esto reduce cada evaluación de $O(n)$ a $O(1)$, un esfuerzo constante sin importar el número de ciudades. Es la razón práctica de que estos métodos puedan hacer millones de evaluaciones sobre instancias grandes.

Por qué no es aplicable el descenso del gradiente

Gran parte de la optimización vista en el curso usa la derivada de la función objetivo: el descenso del gradiente avanza en la dirección de máxima disminución, y las condiciones de Karush-Kuhn-Tucker caracterizan los óptimos mediante gradientes. Aquí nada de eso sirve, por una razón de fondo.

La función $L(\pi)$ no está definida sobre $\mathbb{R}^n$, sino sobre el conjunto Π de permutaciones, que es **discreto**. Entre dos rutas no hay puntos intermedios: no se puede avanzar «un poco» de $(1, 2, 3, 4)$ hacia $(1, 3, 2, 4)$, porque entre ambas no existe nada. La derivada parcial pierde sentido y el gradiente ∇L no existe. Por tanto, no son aplicables ni el descenso del gradiente ni las condiciones KKT, que suponen continuidad y diferenciabilidad.

Esa carencia explica por qué existen las metaheurísticas. Sin una dirección de mejora que indique la derivada, el algoritmo tiene que descubrirla probando: genera vecinos, los evalúa y decide de forma aleatoria controlada cuáles conserva [19, 3]. El contraste es claro: un modelo continuo minimiza su error ajustando parámetros reales; el TSP minimiza un costo combinatorio reordenando una secuencia.

Medición de la calidad de una solución

Como las metaheurísticas no garantizan el óptimo, hace falta medir qué tan buena es la solución. Cuando se conoce el costo óptimo L^*, como en las instancias de referencia de TSPLIB [23], se usa el **gap relativo**:

$$\text{gap}(\,\%) = \frac{L(\pi) - L^*}{L^*} \times 100. \tag{24}$$

Al expresar el exceso en porcentaje, permite comparar instancias de tamaños distintos. La diferencia absoluta no lo permitiría: 500 unidades de más no significan lo mismo con 50 ciudades que con 500.

6. Aplicación Práctica

Una pequeña empresa de distribución reparte desde un almacén central a cinco clientes (A, B, C, D y E). El vehículo debe salir del almacén, visitar cada cliente una sola vez y volver. El objetivo es encontrar el orden de visita que minimice la distancia total. Es, por tanto, un TSP simétrico con $n = 6$ puntos, que se resuelve con **Optimización por Colonia de Hormigas (ACO)**.

La Tabla 7 muestra la matriz de distancias. Los valores son ficticios, en kilómetros, y cumplen $d_{ij} = d_{ji}$.

Tabla 7: Matriz de distancias entre el almacén y los puntos de entrega (km)

	Alm.	**A**	**B**	**C**	**D**	**E**
Alm.	0	6.3	8.6	9.4	7.1	3.6
A	6.3	0	3.2	6.1	7.1	4.1
B	8.6	3.2	0	3.6	6.3	5.4
C	9.4	6.1	3.6	0	4.1	5.8
D	7.1	7.1	6.3	4.1	0	4.1
E	3.6	4.1	5.4	5.8	4.1	0

Como punto de comparación se usa la ruta que sigue el orden convencional de los pedidos:

$$\text{Alm.} \to A \to D \to B \to E \to C \to \text{Alm.}, \qquad L_{\text{referencia}} = 6{,}3+7{,}1+6{,}3+5{,}4+5{,}8+9{,}4 = 40{,}3 \text{ km}.$$

6.1. Justificación del Método Elegido

Se eligió ACO por tres razones, cada una ligada a lo visto en el informe:

1. **Escala mejor que los métodos exactos.** Con 6 puntos hay solo 60 rutas, pero con 20 clientes serían 6×10^{16} (Tabla 2). ACO construye buenas rutas sin revisarlas todas.
2. **Corrige la miopía de las heurísticas.** El vecino más cercano decide solo por cercanía; ACO combina esa misma cercanía (η_{ij}) con la memoria de la colonia (τ_{ij}).
3. **Evita quedarse atascado.** La evaporación impide que la primera ruta aceptable domine la búsqueda.

Como toda metaheurística, ACO no garantiza el óptimo y depende de sus parámetros. Aquí se usa una configuración sencilla, acorde con el tamaño del problema (Tabla 8).

Tabla 8: Parámetros utilizados en la aplicación de ACO

Parámetro	**Valor**	**Función y motivo**
m	6	Hormigas por iteración: una por punto.
α	1	Peso de la feromona: se mantiene la experiencia de la colonia.
β	2	Peso de la cercanía: se le da algo más de importancia que a la feromona.
ρ	0.5	Evaporación: la mitad de la feromona se borra en cada iteración, para evitar estancarse.
Q	100	Constante de depósito de feromona.
$t_{\text{máx}}$	50	Número máximo de iteraciones.

Cómo decide una hormiga en este caso. En la primera iteración toda la feromona es igual, así que la regla (21) depende solo de la cercanía, $(1/d_{ij})^2$. Desde el almacén, una hormiga elige E con probabilidad 52,5 %, A con 17,1 %, D con 13,5 %, B con 9,2 % y C con 7,7 %. Prefiere lo cercano, como el vecino más cercano, pero no siempre: la elección es probabilística y deja espacio para explorar. Al terminar la iteración, la mitad de la feromona se evapora y cada hormiga deposita Q/L en su ruta: una ruta de 24.9 km deja $100/24{,}9 \approx 4{,}02$ en cada arista, y una de 40.3 km solo 2,48. Así, iteración tras iteración, las aristas de las rutas cortas se vuelven más atractivas.

6.2. Impacto en la Estrategia / Negocio

La mejor ruta encontrada por ACO fue

$$\text{Alm.} \to A \to B \to C \to D \to E \to \text{Alm.}, \qquad L_{\text{ACO}} = 6{,}3 + 3{,}2 + 3{,}6 + 4{,}1 + 4{,}1 + 3{,}6 = 24{,}9 \text{ km},$$

y su recorrido inverso es equivalente por ser un problema simétrico.

Verificación del resultado. Como ACO no garantiza el óptimo, conviene comprobarlo. Con seis puntos es posible: por la ecuación (5) hay solo $(6-1)!/2 = 60$ rutas, y la fuerza bruta (Sección 3.1) las evalúa en una fracción de segundo. La Tabla 9 compara los resultados.

Tabla 9: Comparación de rutas: referencia, heurística, ACO y óptimo

Método	**Ruta**	**Distancia (km)**	**Gap**
Ruta de referencia	Alm. → A → D → B → E → C → Alm.	40.3	61.8 %
Vecino más cercano*	Alm. → E → A → B → C → D → Alm.	25.7	3.2 %
ACO	Alm. → A → B → C → D → E → Alm.	24.9	0 %
Óptimo (fuerza bruta)	Misma ruta que ACO (60 rutas evaluadas)	24.9	—

*Desde E hay un empate: A y D están a 4.1 km. Eligiendo A se obtiene esta ruta; eligiendo D, la óptima.

La ruta de ACO es la óptima (gap de 0 %, ecuación (24)). El vecino más cercano se queda a un 3.2 % y, además, su resultado depende de cómo se resuelva un empate: acierta o falla por azar, no por buscar mejor. Es la miopía descrita en la Sección 3.2.

Frente a la ruta de referencia, la reducción de distancia es

$$\text{Reducción}(\,\%) = \frac{L_{\text{referencia}} - L_{\text{ACO}}}{L_{\text{referencia}}} \times 100 = \frac{40{,}3 - 24{,}9}{40{,}3} \times 100 \approx 38{,}21\,\%. \tag{25}$$

Para la empresa, recorrer 15.4 km menos por viaje significa menos combustible, menos tiempo de transporte y menos desgaste del vehículo. Sin embargo, un 38.21 % menos de distancia no equivale a un 38.21 % menos de costo: para estimar el ahorro real habría que incluir el costo por kilómetro, los tiempos de servicio y el tránsito. Con datos reales de ubicaciones, la empresa podría aplicar el mismo procedimiento a su operación diaria.

Finalmente, el caso resume la lógica de todo el informe sobre el TSP. Con pocos clientes, el problema se resuelve de forma exacta; pero cada cliente adicional multiplica el número de rutas (Tabla 2), y con unas decenas de puntos la enumeración deja de ser viable. Ahí es donde ACO aporta valor: el mismo procedimiento aplicado en este ejemplo escala a instancias en las que ya no es posible verificar el óptimo, y entrega rutas de buena calidad en tiempos manejables.

Referencias

[1] Applegate, D. L., Bixby, R. E., Chvátal, V., & Cook, W. J. (2011). *The traveling salesman problem: A computational study*. Princeton University Press.

[2] Bellman, R. (1962). Dynamic programming treatment of the travelling salesman problem. *Journal of the ACM, 9*(1), 61–63. https://doi.org/10.1145/321105.321111

[3] Blum, C., & Roli, A. (2003). Metaheuristics in combinatorial optimization: Overview and conceptual comparison. *ACM Computing Surveys, 35*(3), 268–308. https://doi.org/10.1145/937503.937505

[4] Croes, G. A. (1958). A method for solving traveling-salesman problems. *Operations Research, 6*(6), 791–812. https://doi.org/10.1287/opre.6.6.791

[5] Dantzig, G. B., Fulkerson, D. R., & Johnson, S. M. (1954). Solution of a large-scale traveling-salesman problem. *Operations Research, 2*(4), 393–410. https://doi.org/10.1287/opre.2.4.393

[6] Díaz, A., Glover, F., Ghaziri, H. M., González, J. L., Laguna, M., Moscato, P., & Tseng, F. T. (1996). *Optimización heurística y redes neuronales con aplicaciones a la ingeniería industrial*. Editorial Paraninfo.

[7] Dorigo, M., Maniezzo, V., & Colorni, A. (1996). Ant system: Optimization by a colony of cooperating agents. *IEEE Transactions on Systems, Man, and Cybernetics, Part B, 26*(1), 29–41. https://doi.org/10.1109/3477.484436

[8] Garey, M. R., & Johnson, D. S. (1979). *Computers and intractability: A guide to the theory of NP-completeness*. W. H. Freeman.

[9] Goldberg, D. E. (1989). *Genetic algorithms in search, optimization and machine learning*. Addison-Wesley.

[10] Gracia, M. D., Mar-Ortiz, J., & Laureano-Casanova, O. (2015). Análisis computacional de los problemas del vendedor viajero y patrones de corte. *Ingeniería, Investigación y Tecnología, 16*(1), 59–70. https://doi.org/10.1016/s1405-7743(15)72107-4

[11] Granada-Echeverri, M., & Santa, J. (2016). *Optimización combinatoria: de la teoría a la práctica*. https://www.researchgate.net/publication/331196434

[12] Held, M., & Karp, R. M. (1962). A dynamic programming approach to sequencing problems. *Journal of the Society for Industrial and Applied Mathematics, 10*(1), 196–210. https://doi.org/10.1137/0110015

[13] Johnson, D. S., & McGeoch, L. A. (1997). The traveling salesman problem: A case study in local optimization. En E. H. L. Aarts & J. K. Lenstra (Eds.), *Local search in combinatorial optimization* (pp. 215–310). John Wiley & Sons.

[14] Karp, R. M. (1972). Reducibility among combinatorial problems. En R. E. Miller & J. W. Thatcher (Eds.), *Complexity of computer computations* (pp. 85–103). Plenum Press. https://doi.org/10.1007/978-1-4684-2001-2_9

[15] Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P. (1983). Optimization by simulated annealing. *Science, 220*(4598), 671–680. https://doi.org/10.1126/science.220.4598.671

[16] Land, A. H., & Doig, A. G. (1960). An automatic method of solving discrete programming problems. *Econometrica, 28*(3), 497–520. https://doi.org/10.2307/1910129

[17] Larrañaga, P., Kuijpers, C. M. H., Murga, R. H., Inza, I., & Dizdarevic, S. (1999). Genetic algorithms for the travelling salesman problem: A review of representations and operators. *Artificial Intelligence Review, 13*(2), 129–170. https://doi.org/10.1023/A:1006529012972

[18] Little, J. D. C., Murty, K. G., Sweeney, D. W., & Karel, C. (1963). An algorithm for the traveling salesman problem. *Operations Research, 11*(6), 972–989. https://doi.org/10.1287/opre.11.6.972

[19] Martí, R. (2003). Procedimientos metaheurísticos en optimización combinatoria. *Matemàtiques*, 1(1), 3–62.

[20] Orizaola Molinero, P. (2024). Estudios de algoritmos para el Problema del Viajante: Modelización y comparación del rendimiento [Trabajo de fin de grado, Universidad de Salamanca]. Repositorio GREDOS. https://gredos.usal.es/handle/10366/164079

[21] Osaba Icedo, E. (2020). Diseño e implementación de una meta-heurística multi-poblacional de optimización combinatoria enfocada a la resolución de problemas de asignación de rutas a vehículos [Tesis doctoral]. Universidad de Deusto, Bilbao. https://doi.org/10.48550/arXiv.2003.11393

[22] Papadimitriou, C., & Steiglitz, K. (1982). *Combinatorial Optimization: Algorithms and Complexity.* Prentice-Hall. https://archive.org/details/combinatorialopt0000papa/page/n5/mode/2up

[23] Reinelt, G. (1991). TSPLIB: A traveling salesman problem library. *ORSA Journal on Computing, 3*(4), 376–384. https://doi.org/10.1287/ijoc.3.4.376

[24] Rosenkrantz, D. J., Stearns, R. E., & Lewis, P. M. (1977). An analysis of several heuristics for the traveling salesman problem. *SIAM Journal on Computing, 6*(3), 563–581. https://doi.org/10.1137/0206041

[25] Silva, B., & Torres, L. M. (2014). Acerca de una versión dinámica del problema de la mochila. *Revista Politécnica, 34*(1), 142. https://www.redalyc.org/articulo.oa?id=688773673018

[26] Talbi, E.-G. (2009). *Metaheuristics: From design to implementation.* John Wiley & Sons.